EDA-AGENTS · DEEP DIVE

The Digital Loop

RTL-to-GDS through five agents, one LibreLane flow, three CLI runtimes — including OpenCode.

PRESENTER

Mauricio Montanares

Scientist, IHP

DATE

2026·04·23

DECK

42 slides · 8 parts

SIBLING

analog.html

What you will learn

- The **RTL-to-GDS problem** the digital agents solve, and where LibreLane sits.
- The **five ADK agents** in detail — master + four specialists, one tool allowlist each.
- Four backends: `adk`, `cc_cli`, `litellm`, opencode — where each plugs in.
- **GenericDesign** — the zero-Python-class path from config.yaml to a running agent hierarchy.

PAIRED NOTEBOOKS

`demo/agents_rtl2gds_counter.ipynb`
ProjectManager + 4 specialists; dry-run default, ~10-15 min real.

`demo/agents_digital_autoresearch.ipynb`
DigitalAutoresearchRunner + OpenCode backend; knob sweep.

Eight parts, five-ish minutes each

01 The **digital problem**

02 User-facing **CLI agents**

03 **ProjectManager** + 4 specialists

04 **DigitalAutoresearchRunner** + 4 backends

05 **GenericDesign** — config-only entry

06 Three **CLI runtimes**

07 Skills + **MCP**

08 **Demo** walkthroughs + close

PART 01

The digital problem

RTL-to-GDS through a flow orchestrator. What the agents actually do on top of LibreLane.

RTL-to-GDS — seven phases, 79 steps

Synth	Floor	PDN	Place	CTS	Route	Signoff
YOSYS	OPENROAD	OPENROAD	OPENROAD	OPENROAD	TRITONROUTE	MAGIC/KLAYOUT

WHERE THE AGENTS SIT

The seven phases are *LibreLane's* responsibility. The digital agents sit on top and decide *when to re-run what with what knobs*. They never invoke Yosys, OpenROAD, Magic, KLayout, or Netgen directly — every action goes through a LibreLane step.

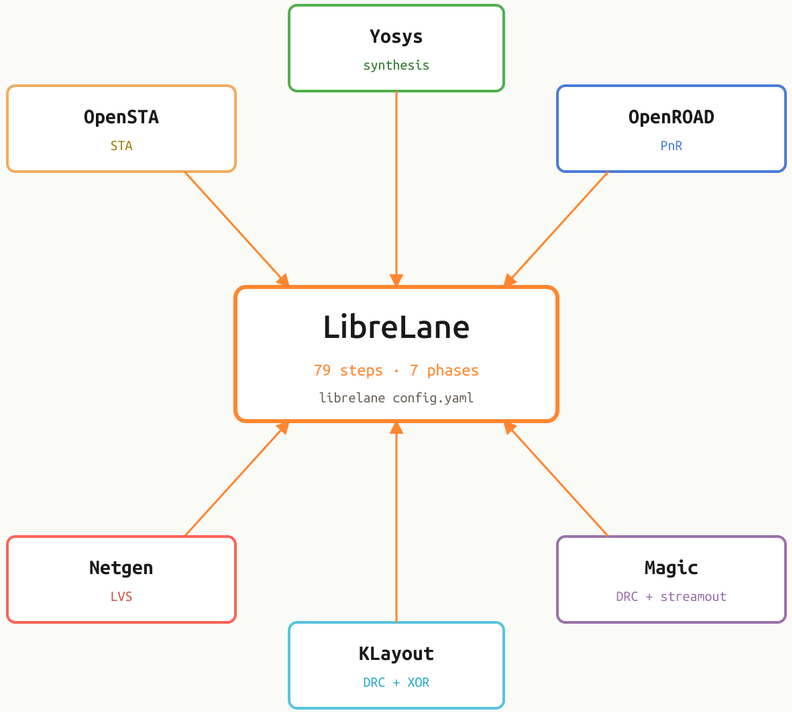
LibreLane is an *orchestrator*, not an EDA tool

Single command: `librelane config.yaml`. Under the hood, six independent open-source projects cooperate.

- **Yosys** — RTL → gate-level netlist.
- **OpenROAD** — floorplan, PDN, placement, CTS, routing.
- **Magic** — DRC + GDS streamout.
- **KLayout** — second DRC engine + XOR cross-check.
- **Netgen** — LVS.
- **OpenSTA** — static timing analysis.

AGENT IMPLICATION

When a flow fails, the fix is in one of the six tools' logs — not in LibreLane itself. Agents diagnose **at the tool layer**.



six independent projects · one orchestrator decides who runs when

Cost asymmetry — why digital autoresearch is discrete

PER LIBRELANE RUN

~ minutes

Small counter: 1–2 min. Chipathon padring: 35–45 min. Magic streamout can dominate on IHP.

PER LLM PROPOSAL

~ cents – $1

Model-dependent. Gemini Flash « Sonnet « Opus. OpenCode multi-provider opens the cheap end.

SEARCH-STRATEGY IMPLICATION

Budget per run: **a handful** of evaluations. `DigitalAutoresearchRunner` defaults to `budget=5`. Design space is *discrete* — density in {60, 65, 70}, clock in {50, 40, 30} ns, PDN pitch in {4, 6, 8} — with nearest-value snapping on proposals.

PART 02

User-facing CLI agents

Three agents. Two drive Docker. One writes config.yaml. All five ship in both Claude Code and OpenCode.

The user-facing agent roster

DIGITAL · HOST-SIDE

`digital-testbench-author`

Authors a cocotb testbench that runs untouched across RTL, post-synth GL, and post-PnR GL + SDF.

DIGITAL · DOCKER

`gf180-docker-digital`

Drives the full RTL-to-GDS flow on GF180MCU inside `hpretl/iic-osic-tools`. User approves each docker-exec call.

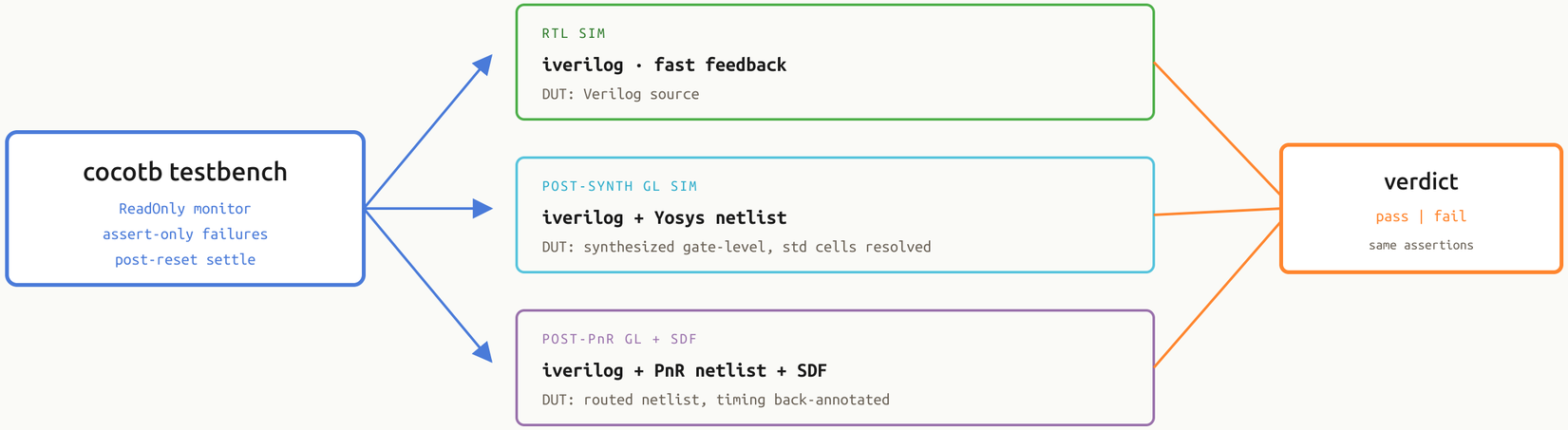
SHARED · CONFIG

`librelane-expert`

config.yaml authoring specialist. Does not run the flow — helps you write it.

All three registered under `.claude/agents/` and mirrored at `.opencode/agent/`. Same prompt body, two runtimes.

digital-testbench-author — one TB, three sims



DISCIPLINE

ReadOnly monitor
no mutation of DUT signals

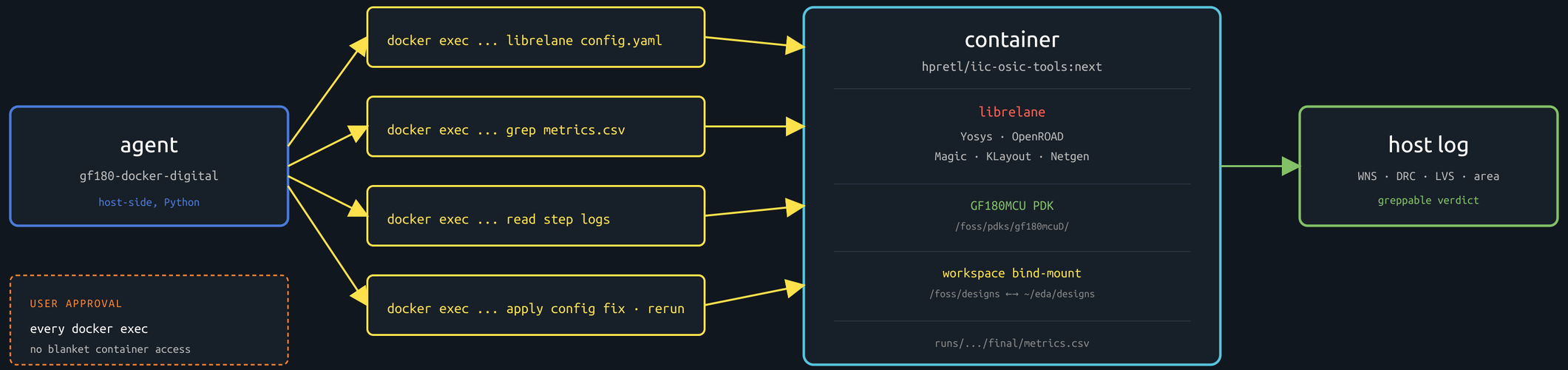
assert-only failures
no "print and continue"

post-reset settle
no false pass on X-prop

same assertions
across all three harnesses

The single biggest lever for digital verification productivity: one testbench, three simulation modes, identical passes/fails.

gf180-docker-digital — the end-to-end driver



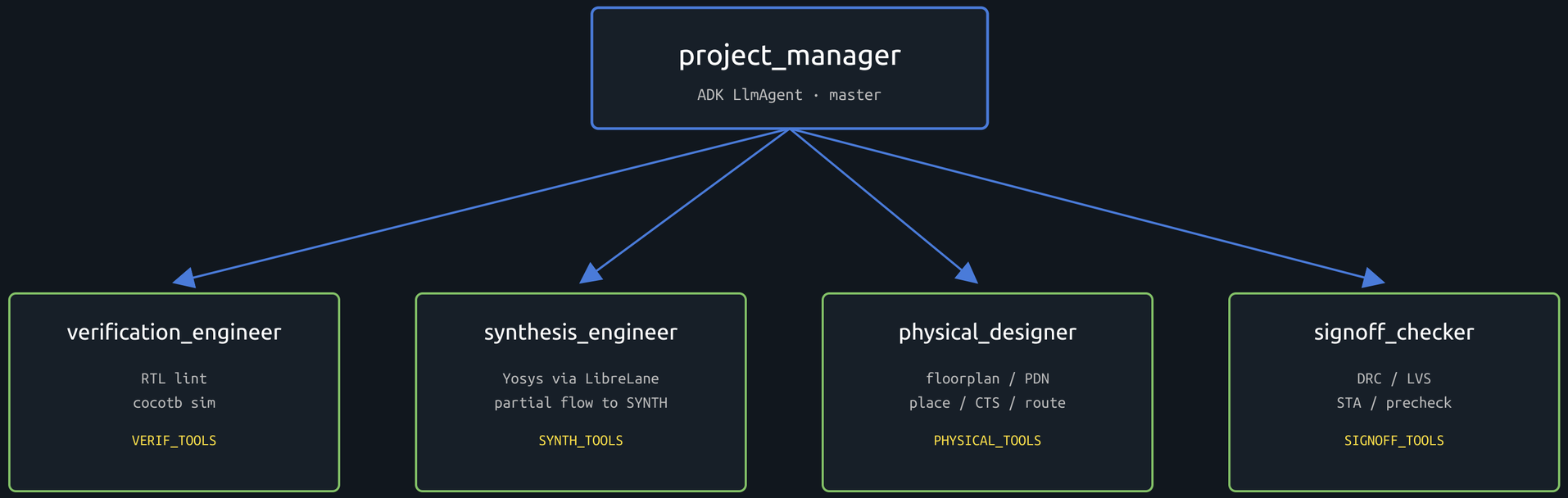
Container is disposable · artifacts persist in the bind-mount · agent reads metrics.csv and iterates with targeted config edits.

PART 03

ProjectManager + 4 specialists

Google ADK multi-agent tree. Master delegates. Each specialist has a bounded tool allowlist.

The ADK tree



Source: `src/eda_agents/agents/digital_adk_agents.py` · *master line 473, specialists 248 / 273 / 302 / 336.*

verification_engineer

verification_engineer

Gates synthesis. If lint or sim fails, the master does not call the next specialist.

```
VERIF_TOOLS = { run_rtl_lint, run_rtl_sim }
```

- **run_rtl_lint** — verilator + yosys lint, error-level diagnostics only.
- **run_rtl_sim** — cocotb + iverilog on the authored testbench.
- No access to LibreLane. No access to DRC / LVS tools. Two tools only; two decisions possible (proceed or fail).

WHERE THIS SPECIALIST SITS IN THE LIBRELANE PIPELINE

RTL/TB THIS	Synth	Floor	PDN	Place	CTS	Route	Signoff

synthesis_engineer

synthesis_engineer

Drives LibreLane partial flow up to the SYNTH step. Reads timing reports, proposes config tweaks, re-runs.

```
SYNTH_TOOLS = { run_librelane_flow, read_timing_report, check_flow_status,
modify_flow_config }
```

- **No physical-design tools** — cannot accidentally invoke OpenROAD placement.
- **Config mutation is scoped** — `modify_flow_config` accepts only synthesis-domain keys.
- Re-run after config change; `read_timing_report` decides if the synth is acceptable.

WHERE THIS SPECIALIST SITS IN THE LIBRELANE PIPELINE

RTL/TB	Synth THIS	Floor	PDN	Place	CTS	Route	Signoff

physical_designer

physical_designer

Floorplan, PDN, placement, CTS, routing. Everything between synthesis and signoff.

```
PHYSICAL_TOOLS = { run_librelane_flow, run_physical_slice, modify_flow_config,
read_timing_report, check_flow_status }
```

- **run_physical_slice** — re-run a single phase (e.g. just CTS) without re-synthesising.
- Iterative by nature — routing often needs two or three passes on tough designs.
- Reads timing at each slice; escalates to master if post-CTS WNS is > |budget|.

WHERE THIS SPECIALIST SITS IN THE LIBRELANE PIPELINE

RTL/TB	Synth	Floor THIS	PDN THIS	Place THIS	CTS THIS	Route THIS	Signoff

signoff_checker

signoff_checker

DRC, LVS, STA, precheck. The last gate. If it passes, the run is accepted.

```
SIGNOFF_TOOLS = { run_klayout_drc, read_drc_summary, run_klayout_lvs,
modify_flow_config, rerun_flow, check_flow_status, run_precheck }
```

- **Two independent DRC engines** — Magic (in LibreLane) + KLayout (here). XOR cross-check catches tool bugs.
- **rerun_flow** — if signoff fails and the root cause is upstream, this specialist can trigger a full re-run.
- **run_precheck** — wafer-space MPW precheck (for Chipathon / TinyTapeout submissions).

WHERE THIS SPECIALIST SITS IN THE LIBRELANE PIPELINE

RTL/TB	Synth	Floor	PDN	Place	CTS	Route	Signoff THIS

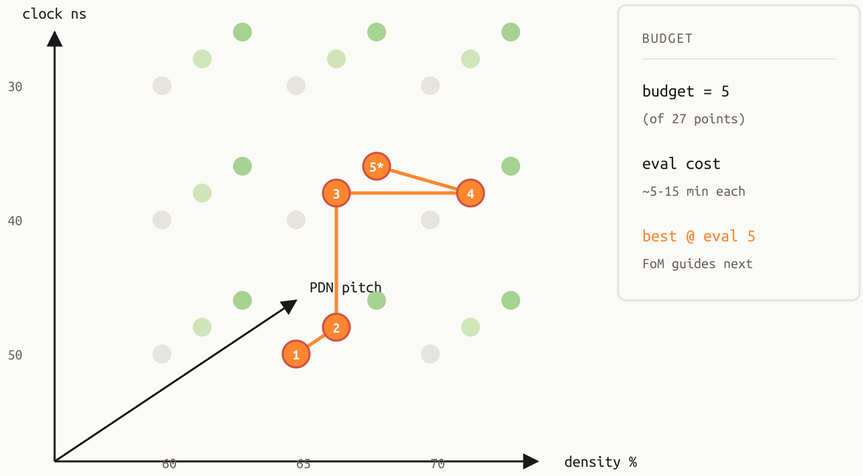
PART 04

DigitalAutoresearch Runner · 4 backends

Greedy search over LibreLane knobs. Where OpenCode plugs in as a first-class backend.

DigitalAutoresearchRunner

DESIGN SPACE · 27 POINTS
density × clock period × PDN pitch



BUDGET

budget = 5
(of 27 points)

eval cost
~5-15 min each

best @ eval 5
FoM guides next

FoM = timing_w · WNS + area_w · area + power_w · power

nearest-value snap on each proposal · greedy refinement with restart-on-regression

- **Same greedy shape** as analog — Propose → Map → Evaluate → Score → Log → Iterate.
- **Discrete space.** density ∈ {60, 65, 70}, clock ∈ {50, 40, 30} ns, PDN pitch ∈ {4, 6, 8}.
- **Budget 5** by default. Each eval = full LibreLane run.
- **FoM:** weighted timing (WNS), area, power.

CONSTRUCTOR · BACKENDS ACCEPTED

```
DigitalAutoresearchRunner(
  design=design,
  backend="opencode",
  # adk | cc_cli | litellm | opencode
  budget=5,
)
```

Four backends, one runner

adk
Google ADK runner. 5-agent tree, locked-down tool sets. CI-friendly.

cc_cli
claude --print --output-format json. Anthropic subscription.

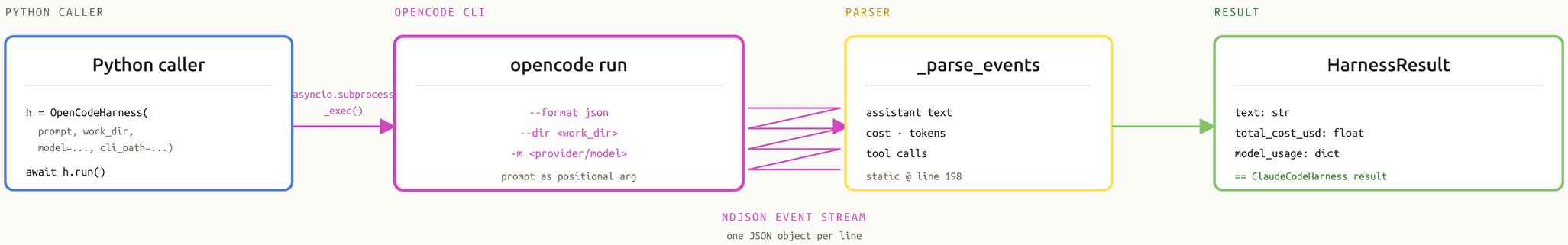
litellm
Plain LiteLLM API call. No CLI, no agent hierarchy. Deterministic + cheap.

opencode
opencode run --format json. Multi-provider: Anthropic, OpenAI, Gemini, Z.AI, Ollama.

BACKEND CHOICE IS A STRING

```
DigitalAutoresearchRunner(design, backend="opencode", opencode_model="openrouter/google/gemini-3-flash-preview")
```

OpenCodeHarness — the plumbing



-m <PROVIDER/MODEL> · PICKED PER CALL, NOT PINNED IN AGENT

anthropic/claude-sonnet-4-6	zai-coding-plan/glm-5.1
openrouter/google/gemini-3-flash	google/gemini-2.5-pro
openai/gpt-5-mini	ollama/llama3.1 (local)

SAME HARNESS · SWAP PROVIDER BY STRING

~263 LOC at `src/eda_agents/agents/opencode_harness.py`
returns the shared HarnessResult so agent code is backend-agnostic

Caveat — `ProjectManager` vs OpenCode

PROJECTMANAGER TODAY

Accepts `backend="adk"` or `backend="cc_cli"`. Raises `ValueError` on anything else. This is deliberate phasing, not an oversight.

OPENCODE PATH TODAY

End-to-end RTL-to-GDS via OpenCode:
`opencode --agent gf180-docker-digital` in the TUI.
Knob-search autoresearch via OpenCode:
`DigitalAutoresearchRunner(backend="opencode")` in Python.

Wiring OpenCode into the ProjectManager multi-agent tree is planned; see `docs/s12_handoffs/s12d_opencode_wrapper.md`.

PART 05

GenericDesign — zero Python class

Any LibreLane config becomes a 13-method DigitalDesign in two lines.

`GenericDesign` — what it wraps

- Takes a LibreLane `config.yaml` path and an optional `pdk_root`.
- Derives the 13 `DigitalDesign` methods the agents expect: project_name, specs_description, fom_description, validate_clone, etc.
- Supports **FoM weight overrides** without subclassing.
- Source: `src/eda_agents/core/designs/generic.py`.

INSTANTIATION

```
from eda_agents.core.designs.generic import GenericDesign

design = GenericDesign(
    config_path="./counter_project/config.yaml",
    pdk_root=None,          # falls back to $PDK_ROOT
    pdk_config="gf180mcu",
    fom_weights={"timing_w": 1.0,
                 "area_w":   0.5,
                 "power_w":  0.3},
)
```

Generic → ProjectManager → flow

```python
import asyncio
from eda_agents.core.designs.generic import GenericDesign
from eda_agents.agents.digital_adk_agents import ProjectManager

design = GenericDesign(config_path="./counter_project/config.yaml",
                       pdk_config="gf180mcu")

pm = ProjectManager(design=design, model="google/gemini-3-flash-preview",
                    backend="cc_cli", max_budget_usd=1.00)

# Dry-run first (safe, no Docker, no LLM)
result = asyncio.run(pm.run(Path("./results"), dry_run=True))

# Real run when ready
result = asyncio.run(pm.run(Path("./results")))
```

This is the entire glue. No registry to edit, no bespoke Python class. Works for any LibreLane-compatible design.

When to subclass `DigitalDesign` explicitly

- **Design fetch hooks** — nix-shell, submodule update, git-lfs, custom RTL generators.
- **Pre-flow RTL prep** — Verilog generation from templates, parameter sweeps.
- **Post-flow metric extraction** — custom metrics beyond metrics.csv.
- **Multi-macro designs** — when one config does not cover the whole project.

SHIPPED EXAMPLES

`fazyrv_hachure.py` — GF180 RISC-V SoC, uses nix-shell.
`systolic_mac_dft.py` — CI fixture, simpler.
Both at `src/eda_agents/core/designs/`. ~150 lines each — explicit is small.

PART 06

Three CLI runtimes

Same agents, same skills. Claude Code, OpenCode, or direct Python — pick by subscription and mode.

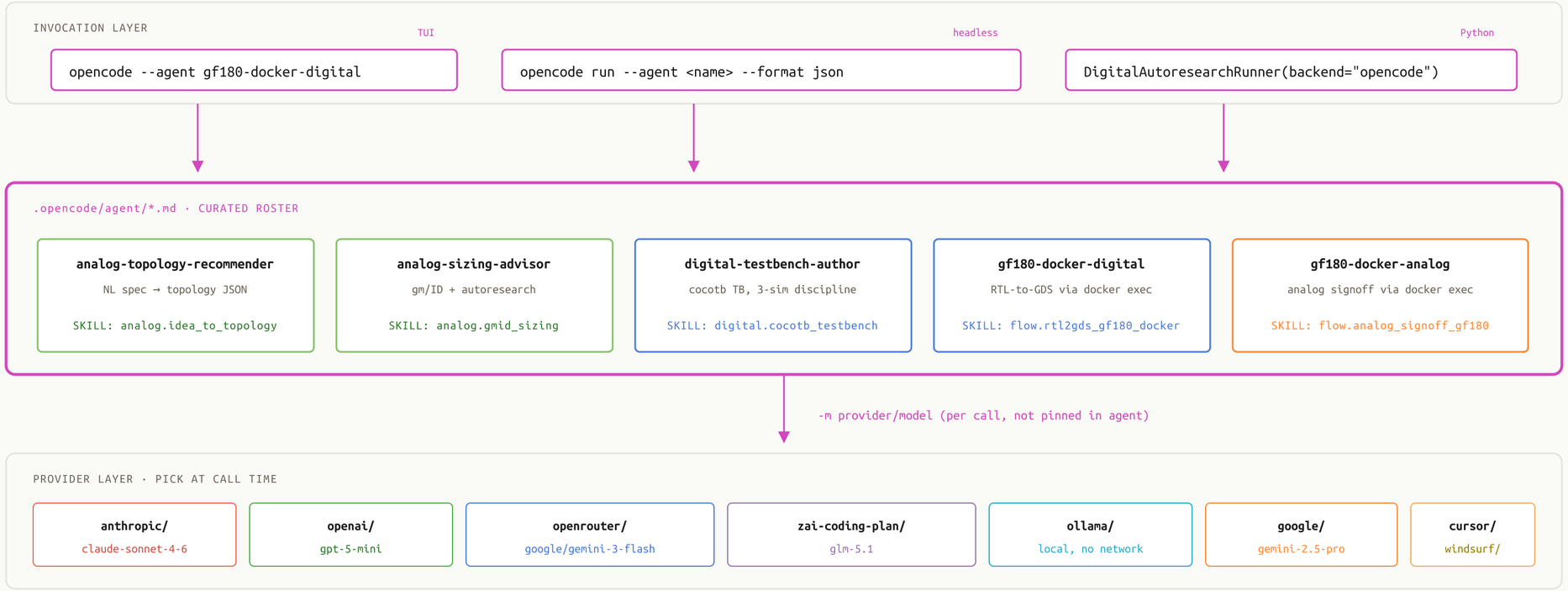
Claude Code — `claude --print`

- **Subscription:** Anthropic only.
- **Interactive:** `/agents gf180-docker-digital` inside a Claude Code session.
- **Headless:** `claude --print --output-format json` wrapped by `ClaudeCodeHarness` in Python.
- **Safety:** `--dangerously-skip-permissions` is double-gated — constructor + env var.

MCP REGISTRATION

```
# ~/.config/claude-code/mcp.json
{"mcpServers": {
  "eda-agents": {
    "type": "stdio",
    "command": "eda-mcp"
}}}
```

OpenCode — one CLI, five agents, any provider



Same 5-agent surface (mirror of `.claude/agents/`) · provider picked per call with `-m` · MCP wired via `./opencode.json` at the repo root.

Direct Python — ADK / DigitalAutoresearchRunner

- **No CLI, no TUI.** Import, construct, await.
- **Deterministic:** the model string is pinned in code.
- **CI-friendly:** no user-in-the-loop approvals.
- This is the path `examples/09_rtl2gds_digital.py` takes with `--backend adk`.

TYPICAL INVOCATION

```
# CI: ProjectManager over ADK
from eda_agents.agents.digital_adk_agents import ProjectManager

pm = ProjectManager(design=design,
                    backend="adk",
                    model="google/gemini-3-flash-preview")
result = await pm.run(Path("./ci_results"))

# Greedy knob search with any of 4 backends
runner = DigitalAutoresearchRunner(
    design=design, backend="opencode",
    opencode_model="openrouter/google/gemini-3-flash-preview",
    budget=5,
)
```

Runtime comparison — which one should you use?

	CLAUDE CODE	OPENCODE	ADK / PYTHON
Subscription	Anthropic	`any provider`	any LiteLLM key
Interactive	`/agents <name>`	`opencode --agent`	—
Headless	`claude --print`	`opencode run`	Python import
Python wrapper	`ClaudeCodeHarness`	`OpenCodeHarness`	native
ProjectManager	yes	TUI only	yes
DigitalAutoresearchRunner	yes	yes	yes
Typical use	Claude subscribers	cheap exploration	CI + benchmarks

PART 07

Skills + MCP

How every backend talks to the same stack.

The `digital.*` namespace

ONE PER ROLE

```
digital.project_manager
digital.verification
digital.synthesis
digital.physical
digital.signoff
```

CRAFT + CRITIQUE

```
digital.cocotb_testbench
digital.critique_sim_failure
digital.critique_synth_lint
```

Source: `src/eda_agents/skills/digital.py`. Each skill is a decorator-registered Markdown template; the MCP server renders on demand.

The `flow.*` namespace — cross-domain bridges

RUNNER

- `flow.runner` — generic LibreLane driver.
- `flow.rtl2gds_gf180_docker` — full GF180 recipe.
- `flow.analog_signoff_gf180_docker` — analog counterpart.

CHECKERS

- `flow.drc_checker` — Magic or KLayout.
- `flow.lvs_checker` — Netgen.

FIXERS

- `flow.drc_fixer` — propose targeted fix, not blanket retry.

A DRC skill does not care whether the failed block is analog or digital — it reads the violation, suggests the fix. That's why these live outside either namespace.

MCP registration — one server, two clients

CLAUDE CODE · ~/.CONFIG/CLAUDE-CODE/MCP.JSON

```
{"mcpServers": {
  "eda-agents": {
    "type": "stdio",
    "command": "eda-mcp"
  }
}}
```

OPENCODE · ./OPENCODE.JSON

```
{"$schema": "https://opencode.ai/config.json",
 "mcp": {"eda-agents": {
    "type": "local",
    "command": [".venv/bin/python",
                "-m", "eda_agents.mcp"]
 }}}
```

BOOTSTRAP

`eda-init` writes both files plus the `.opencode/agent/` roster. Run it once after you clone.

PART 08

Demos · run it yourself

Two notebooks. Safe defaults. Same 4-bit counter on GF180MCU.

Demo 1 — `agents_rtl2gds_counter.ipynb`

SIX STEPS

1. Editable install — `pip install -e .`
2. Docker + GF180 image + CLI + API-key check
3. Stage `counter.v` + `config.yaml` inline
4. Construct `ProjectManager` + dry-run (safe)
5. Full RTL-to-GDS run (gated by `RUN_REAL=True`, ~10–15 min)
6. Read `rtl2gds_results.json` + `metrics.csv`

SAME COUNTER AS THE SIBLING

Uses identical RTL + `config.yaml` as `../../rtl2gds-gf180-docker/demo/rtl2gds_counter.ipynb`. One test design, two tutorials.

Demo 2 — `agents_digital_autoresearch.ipynb`

WHAT IT SHOWCASES

1. `DigitalAutoresearchRunner` on the same counter design.
2. Knob sweep over density (60 / 65 / 70) and clock period (50 / 40 / 30 ns).
3. Backend opencode with `-m openrouter/google/gemini-3-flash-preview`.
4. Demonstrates resumable runs — kill the kernel, re-run, the TSV picks up.

WHY OPENCODE HERE

Five LibreLane evaluations at Gemini-Flash-via-OpenRouter prices is well under $0.50. The same run on Sonnet is 10–20× that. Multi-provider is an *economic* feature for greedy loops.

Prerequisites summary

PATH	INSTALL	CREDENTIALS
Docker flow	`docker pull hpretl/iic-osic-tools:next`	—
cc_cli backend	`npm install -g @anthropic-ai/claude-code`	Anthropic subscription
opencode backend	`npm install -g opencode-ai`	any provider key (via -m)
ADK backend	`pip install "eda-agents[adk]"`	LiteLLM-compatible key
MCP server	`pip install -e .` · `eda-init`	—

All five paths coexist in the same clone. Flip a backend string to switch.

Safety — notebooks default to rehearse

- Every `RUN_*` flag defaults to `False`.
- Opening a notebook runs **no Docker, no LLM, no cost**.
- Each flip is a deliberate decision. A teaching session can step through the whole story without committing to a paid run.
- Identical policy on both analog and digital notebooks.

BUDGET GUARD

`ProjectManager(max_budget_usd=1.00)` is the hard cap for `cc_cli` runs. Exceed it and the flow stops cleanly with a surfaced verdict.

Starter commands — cheat sheet

```
# From the repo root, with venv activated + eda-init already run:

# Claude Code · interactive
/agents gf180-docker-digital

# OpenCode · interactive, Gemini Flash via OpenRouter
opencode --agent gf180-docker-digital \
  -m openrouter/google/gemini-3-flash-preview

# OpenCode · headless, Z.AI GLM-5.1
opencode run --agent gf180-docker-digital \
  -m zai-coding-plan/glm-5.1 \
  "Harden counter under ./counter_project/"

# Python · greedy knob search via OpenCode backend
python examples/10_digital_autoresearch.py \
  --backend opencode \
  --opencode-model openrouter/google/gemini-3-flash-preview \
  --design counter_project --budget 5
```

DIGITAL · THANK YOU

The digital loop is infrastructure.

LibreLane is the flow engine. ProjectManager is the hierarchy. GenericDesign is the entry. Three CLI runtimes. Four backends. One codebase.

`github.com/Mauricio-xx/eda-agents` · *sibling deck:* `analog.html`